SS 15

Machine Learning II - Structured Models

Exercise sheet **Lagrange Dual to the LP Relaxation. LP (TRWS) vs. Naïve Solver.**

Bogdan Savchynskyy - Bogdan.Savchynskyy@tu-dresden.de

Exercise 1 (LP Tight Sase)**.** Download and unpack `color-seg-n4` datasets from the OpenGM benchmark `http://hci.iwr.uni-heidelberg.de/opengm2/`). Download an update of the python scripts to the exercise.

Non-Reparametrized problem Run an LP solver (TRWS) on the models to find those, which are LP-tight (bound=value). Check that e.g. `lake-small` and `crops-small` are such models.

```
opengm_min_sum -a TRWS --energy TL1 --maxIt 1000 -o crops-trws.h5 -p 1 -v  -m crops-small.h5
opengm_min_sum -a TRWS --energy TL1 --maxIt 1000 -o lake-trws.h5 -p 1 -v  -m lake-small.h5
```

Plot the resulting labeling using `show_labeling.py`:

```
python show_labeling.py -V 240 -H 360 -v -F -f crops-trws.h5
python show_labeling.py -V 240 -H 360 -v -F -f lake-trws.h5
```

These models are color segmentation models, where the input image is approximated with a small predefined number of colors. Unary factor are color differences between the input image and the predefined colors. Pairwise factors are *Potts potentials* (also known as *multilabel total variation*): $\theta_{uv}(x_u, x_v) = \begin{cases} 0, & x_u = x_v \\ \alpha > 0, & x_u \neq x_v \end{cases}$. However the colors used for approximation are not stored in the model (e.g. `crops-small.h5`) and hence can not be reproduced by `show_labeling.py`. The script outputs random colors.

Solve the problems with the `naive_solver.py`, which outputs the labeling corresponding to the lowest unary potentials independently of each other.

```
python naive_solver.py crops-small.h5
cp naive_labeling.h5 crops-naive.h5
python naive_solver.py lake-small.h5
cp naive_labeling.h5 lake-naive.h5
```

Check that the returned energy value is significantly bigger and the bound is smaller than those returned by TRWS. Check the corresponding images with

```
python show_labeling.py -V 240 -H 360 -v -F -f crops-naive.h5
python show_labeling.py -V 240 -H 360 -v -F -f lake-naive.h5
```

They look significantly less smooth than those corresponding to TRWS, since pairwise factors do not participate in the decision making.

Reparametrised Problem. Run the command line tool to solver the dual and save the reparametrised models (there is one solver, which implicitly allows to save a reparametrised model - `CombiLP`):

```
opengm_min_sum -a CombiLP --maxIt 1000 -m crops-small.h5 -p 1 -v --savedfilename crops-repa.h5
opengm_min_sum -a CombiLP --maxIt 1000 -m lake-small.h5 -p 1 -v --savedfilename lake-repa.h5
```

Solve the problems with the `naive_solver.py`, which outputs the labeling corresponding to the lowest unary potentials independently of each other.

```
python naive_solver.py crops-repa.h5
cp naive_labeling.h5 crops-repa-naive.h5
python naive_solver.py lake-repa.h5
cp naive_labeling.h5 lake-repa-naive.h5
```

Check that the returned energy value and bound are exactly the same as those returned by TRWS. Check the corresponding images with

```
python show_labeling.py -V 240 -H 360 -v -F -f crops-repa-naive.h5
python show_labeling.py -V 240 -H 360 -v -F -f lake-repa-naive.h5
```

and compare them to those returned by TRWS. They look the same, do not they?

Exercise 2 (Non-LP Tight Case)**.** Repeat the same experiment with `tsu-gm` instance from the `mrf-stereo` dataset:

```
#1.  Get TRWS bound and value. Do they differ from each other?
 opengm_min_sum -a TRWS --energy TL2 --maxIt 300 -o tsu-trws.h5 -p 1 -v  -m tsu-gm.h5
#2.  Computing close-to-optimal reparametrization
 opengm_min_sum -a CombiLP --maxIt 100 -p 1 -v  -m tsu-gm.h5
                                  --maxNumberOfILPCycles 0 --savedfilename tsu-repa.h5
#3.  Run naive solver on the original model ...
 python naive_solver.py tsu-gm.h5
 cp naive_labeling.h5 tsu-naive.h5
#4. ... and reparametrized model
 python naive_solver.py tsu-repa.h5
 cp naive_labeling.h5 tsu-repa-naive.h5
# compare values and bounds

#5. Check how the results differ graphically:
python show_labeling.py -V 288 -H 384 -v -C -f tsu-trws.h5
python show_labeling.py -V 288 -H 384 -v -C -f tsu-repa-naive.h5
python show_labeling.py -V 288 -H 384 -v -C -f tsu-naive.h5

#6. Repeat 1-5  with more iterations for TRWS and reparametrizer (CombiLP),
# i.e. use '--maxIt 300' instead of '--maxIt 100' in items 1 and 2
```